

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего
образования
«**Национальный исследовательский университет ИТМО**»
Факультет Программной инженерии и компьютерной техники

Лабораторная работа №1
по дисциплине «Сервис-ориентированная архитектура»

Вариант: 67402

Преподаватель:
Коновалов Арсений Антонович

Выполнил:
Бондаренко Артем Андреевич
Федоров Евгений Константинович

Санкт-Петербург, 2026

Оглавление

Задание	3
Выполнение.....	4
Выводы.....	5

Задание

Введите вариант:

67402

Внимание! У разных вариантов разный текст задания!

Разработать спецификацию в формате OpenAPI для набора веб-сервисов, реализующего следующую функциональность:

Первый веб-сервис должен осуществлять управление коллекцией объектов. В коллекции необходимо хранить объекты класса `Vehicle`, описание которого приведено ниже:

```
public class Vehicle {
    private Long id; //Поле не может быть null, Значение поля должно быть больше 0, Значение этого поля должно быть уникальным, Значение этого поля должно генерироваться автоматически
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Coordinates coordinates; //Поле не может быть null
    private java.util.Date creationDate; //Поле не может быть null, Значение этого поля должно генерироваться автоматически
    private double enginePower; //Значение поля должно быть больше 0
    private VehicleType type; //Поле не может быть null
    private FuelType fuelType; //Поле не может быть null
}

public class Coordinates {
    private Double x; //Поле не может быть null
    private long y; //Значение поля должно быть больше -575
}

public enum VehicleType {
    PLANE,
    HELICOPTER,
    SPACESHIP;
}

public enum FuelType {
    ELECTRICITY,
    MANPOWER,
    ANTIMATTER;
}
```

Веб-сервис должен удовлетворять следующим требованиям:

- API, реализуемый сервисом, должен соответствовать рекомендациям подхода RESTful.
- Необходимо реализовать следующий базовый набор операций с объектами коллекции: добавление нового элемента, получение элемента по ИД, обновление элемента, удаление элемента, получение массива элементов.
- Операция, выполняемая над объектом коллекции, должна определяться методом HTTP-запроса.
- Операция получения массива элементов должна поддерживать возможность сортировки и фильтрации по любой комбинации полей класса, а также возможность分页ного вывода результатов выборки с указанием размера и порядкового номера выводимой страницы.
- Все параметры, необходимые для выполнения операции, должны передаваться в URL запроса.
- Информация об объектах коллекции должна передаваться в формате xml.
- В случае передачи сервису данных, нарушающих заданные на уровне класса ограничения целостности, сервис должен возвращать код ответа http, соответствующий произошедшей ошибке.

Помимо базового набора, веб-сервис должен поддерживать следующие операции над объектами коллекции:

- Рассчитать сумму значений поля enginePower для всех объектов.
- Сгруппировать объекты по значению поля id, вернуть количество элементов в каждой группе.
- Вернуть массив объектов, значение поля type которых больше заданного.

Эти операции должны размещаться на отдельных URL.

Второй веб-сервис должен располагаться на URL `/shop`, и реализовывать ряд дополнительных операций, связанных с вызовом API первого сервиса:

- `/search/by-type/{type}` : найти все транспортные средства заданного типа
- `/search/by-engine-power/{from}/{to}` : найти все транспортные средства с мощностью двигателя в заданном диапазоне

Эти операции также должны размещаться на отдельных URL.

Для разработанной спецификации необходимо сгенерировать интерактивную веб-документацию с помощью Swagger UI. Документация должна содержать описание всех REST API обоих сервисов с текстовым описанием функциональности каждой операции. Созданную веб-документацию необходимо развернуть на сервере `helios`.

Выполнение

Исходный код: <https://github.com/DopleZZ/SOA>

Сгенерированный Swagger UI: <https://se.ifmo.ru/~s409753/swagger-ui.html>

Vehicle Collection Service

1.0.0

OpenAPI 3.0

Vehicle service.yaml

Proprietary

Servers

{scheme}://{host}:{port}

Computed URL: http://localhost:8080

Server variables

scheme

http

host

localhost

port

8080

Vehicles

CRUD и выборки над коллекцией Vehicle

POST

/api/vehicles

Добавить новый элемент в коллекцию

GET

/api/vehicles

Получить массив элементов коллекции

GET

/api/vehicles/{id}

Получить элемент по ID

PUT

/api/vehicles/{id}

Обновить элемент коллекции

DELETE

/api/vehicles/{id}

Удалить элемент коллекции

GET

/api/vehicles/sum-engine-power

Расчитать сумму значений поля enginePower для всех объектов коллекции

GET

/api/vehicles/group-count-by-id

Сгруппировать объекты по значению поля id и вернуть количество элементов в каждой группе

GET

/api/vehicles/by-type-greater/{type}

Вернуть массив объектов, значение поля type которых больше заданного

Schemas

VehicleType

FuelType

Shop Service

1.0.0

OpenAPI 3.0

Shop service.yaml

Дополнительный сервис, реализующий операции поиска транспортных средств, делегируя обращения к API первого сервиса (vehicle-service). Все операции возвращают данные в формате XML.
Proprietary

Servers

{scheme}://{host}:{port}

Computed URL: http://localhost:8081

Server variables

scheme

http

host

localhost

port

8081

Shop

операции поиска по коллекции Vehicle через vehicle-service

GET

/shop/search/by-type/{type}

Найти все транспортные средства заданного типа

GET

/shop/search/by-engine-power/{from}/{to}

Найти все транспортные средства с мощностью двигателя в заданном диапазоне

Schemas

VehicleType

Coordinates

Vehicle

FuelType

Error

Выводы

В ходе выполнения лабораторной разрабатывать спецификацию в формате OpenAPI.
Научились генерировать Swagger для набора спецификаций